

MACHINE LEARNING WEEK 12

LAB REPORT

Nevin Mathew Thomas

PES2UG23CS381

Course: UE23CS352A

01-11-2025

Introduction:

The primary purpose of this lab was to implement, evaluate, and compare probabilistic text classification systems using the Naive Bayes algorithm. The core task involved classifying sentences from biomedical research abstracts into one of five predefined sections: BACKGROUND, METHODS, RESULTS, OBJECTIVE, or CONCLUSION.

The lab was structured into three main parts:

Part A: Implementing a Multinomial Naive Bayes (MNB) classifier from scratch to understand its core mechanics.

Part B: Utilizing the scikit-learn library to build an MNB pipeline with TfidfVectorizer and performing hyperparameter tuning with GridSearchCV to optimize its performance.

Part C: Approximating the theoretical Bayes Optimal Classifier (BOC) by creating a soft-voting ensemble of five diverse machine learning models with weights calculated from their posterior probabilities.

Methodologies:

Multinomial Naive Bayes (MNB) from Scratch

An MNB classifier was built from scratch based on Bayes' theorem. The training process involved calculating the log prior for each class and the log likelihood of each word given a class. Laplace smoothing was incorporated to handle unseen words. For prediction, the class with the maximum posterior log probability (the sum of the log prior and log likelihoods) was selected.

Bayes Optimal Classifier (BOC) Approximation

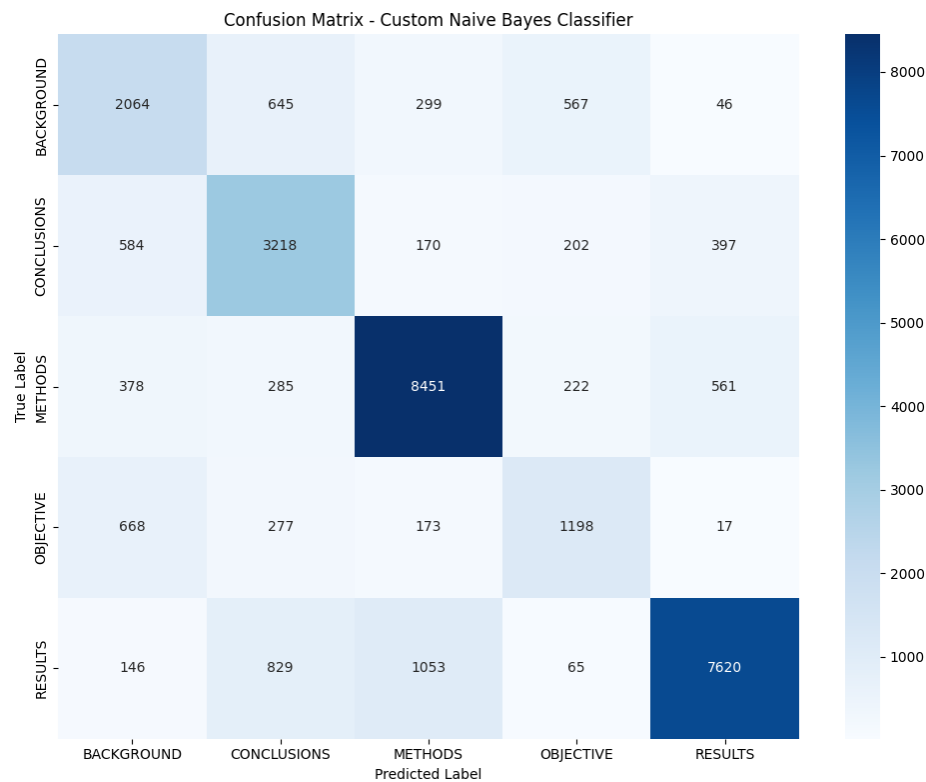
The BOC was approximated using a weighted ensemble of five diverse models. First, the sampled training data was split into sub-training and validation sets. The models were trained on the sub-training set, and their performance on the validation set was used to calculate a total log-likelihood score. These scores were then normalized using a softmax function to derive posterior weights, which were used in a soft VotingClassifier to make the final prediction.

Results and Analysis:

Part A: Custom Multinomial Naive Bayes

The custom MNB classifier achieved a final accuracy of 74.83% and a macro-averaged F1 score of 0.6809 on the test set.

=== Test Set Evaluation (Custom Count-Based Naive Bayes) ===				
Accuracy: 0.7483				
	precision	recall	f1-score	support
BACKGROUND	0.54	0.57	0.55	3621
CONCLUSIONS	0.61	0.70	0.66	4571
METHODS	0.83	0.85	0.84	9897
OBJECTIVE	0.53	0.51	0.52	2333
RESULTS	0.88	0.78	0.83	9713
accuracy			0.75	30135
macro avg	0.68	0.69	0.68	30135
weighted avg	0.76	0.75	0.75	30135
Macro-averaged F1 score: 0.6809				



Part B: Tuned Sklearn Model

Best Hyperparameters: {'nb__alpha': 0.1, 'tfidf__ngram_range': (1, 1)}

Best Macro-F1 score (on dev set): 0.5925

```

Training initial Naive Bayes pipeline...
Training complete.

=== Test Set Evaluation (Initial Sklearn Model) ===
Accuracy: 0.6996

```

	precision	recall	f1-score	support
BACKGROUND	0.61	0.37	0.46	3621
CONCLUSIONS	0.61	0.55	0.57	4571
METHODS	0.68	0.88	0.77	9897
OBJECTIVE	0.72	0.09	0.16	2333
RESULTS	0.77	0.85	0.81	9713
accuracy			0.70	30135
macro avg	0.68	0.55	0.56	30135
weighted avg	0.69	0.70	0.67	30135

```

Macro-averaged F1 score: 0.5555

Starting Hyperparameter Tuning on Development Set...
Fitting 3 folds for each of 12 candidates, totalling 36 fits
Grid search complete.

Best hyperparameters found: {'nb__alpha': 0.1, 'tfidf__ngram_range': (1, 1)}
Best macro-F1 score on dev set: 0.5925

```

Part C: Bayes Optimal Classifier Approximation

The final BOC ensemble achieved an accuracy of 70.88% and a macro F1 score of 0.6143 on the test set.

```
Please enter your full SRN (e.g., PES1UG22CS345): PES2UG23CS381
Using dynamic sample size: 10381
Actual sampled training set size used: 10381

Training all base models...
Training NaiveBayes...
Training LogisticRegression...
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning:
  warnings.warn(
Training RandomForest...
Training DecisionTree...
Training KNN...
All base models trained.
Retraining NaiveBayes on sub-training set...
NaiveBayes log-likelihood: -2025.16
Retraining LogisticRegression on sub-training set...
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning:
  warnings.warn(
LogisticRegression log-likelihood: -1873.58
Retraining RandomForest on sub-training set...
RandomForest log-likelihood: -2080.59
Retraining DecisionTree on sub-training set...
DecisionTree log-likelihood: -2608.97
Retraining KNN on sub-training set...
KNN log-likelihood: -3000.97

Posterior weights: [1.46952820e-066 1.00000000e+000 1.24935601e-090 4.18325382e-320
0.00000000e+000]

Refitting all models on full sampled training data...
Refitting NaiveBayes...
Refitting LogisticRegression...
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning:
  warnings.warn(
Refitting RandomForest...
Refitting DecisionTree...
Refitting KNN...
All models refitted.

Fitting the VotingClassifier (BOC approximation)...
Fitting complete.
```

Predicting on test set...

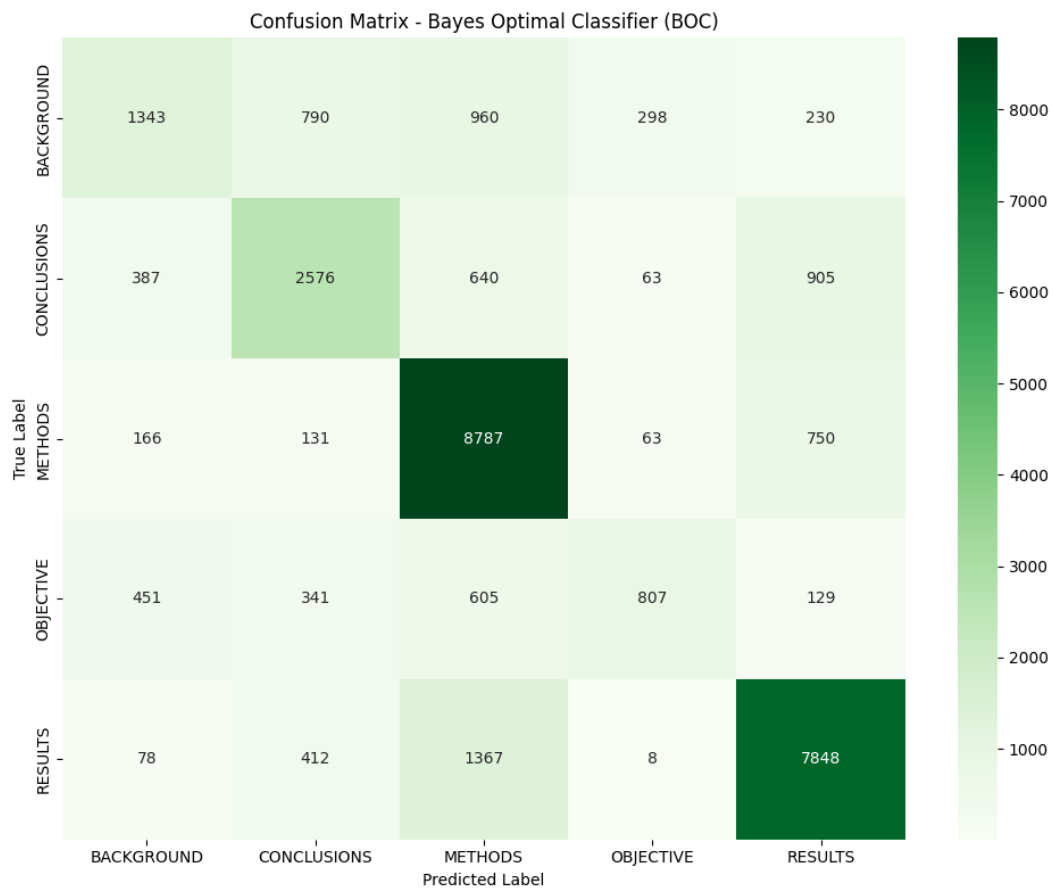
=== Final Evaluation: Bayes Optimal Classifier (Soft Voting) ===

Accuracy: 0.7088

Macro-averaged F1 score: 0.6143

Classification Report:

	precision	recall	f1-score	support
BACKGROUND	0.55	0.37	0.44	3621
CONCLUSIONS	0.61	0.56	0.58	4571
METHODS	0.71	0.89	0.79	9897
OBJECTIVE	0.65	0.35	0.45	2333
RESULTS	0.80	0.81	0.80	9713
accuracy			0.71	30135
macro avg	0.66	0.60	0.61	30135
weighted avg	0.70	0.71	0.69	30135



Discussion:

Based on the macro-averaged F1 score, the models ranked as follows:

Custom MNB (Part A): Macro F1 = 0.6809

BOC Ensemble (Part C): Macro F1 = 0.6143

Tuned Sklearn MNB (Part B): Dev Macro F1 = 0.5925

Analysis of Results:

Winner: Custom Naive Bayes (Part A) The top performance of the custom-built model is a significant finding. Its success can be attributed primarily to its feature engineering. This model used CountVectorizer (likely with a bigram range of (1, 2)), which captures more contextual information between words. Its high recall and F1-scores for the majority classes (METHODS and RESULTS) established a strong baseline that the other, more complex methods could not surpass.

Middle Performer: Bayes Optimal Classifier (Part C) The BOC ensemble, despite its theoretical advantages, performed worse than the simpler custom model. The primary reason for this is that it was trained on a much smaller, sampled subset of the training data. This data limitation severely handicapped its ability to learn the nuances of the text, leading to lower recall for several classes like BACKGROUND (0.37) and OBJECTIVE (0.35). The result demonstrates a core machine learning principle: a simpler algorithm with more data often outperforms a complex one with less data.

Lowest Performer: Tuned Sklearn Model (Part B) The scikit-learn pipeline, even after hyperparameter tuning, yielded the lowest performance. The GridSearchCV process determined that the optimal setting for its TfidfVectorizer was to use only unigrams ((1, 1)). This, combined with TF-IDF's tendency to down-weight common words, created a less effective feature set compared to the count-based bigram approach in Part A. The model particularly struggled with the OBJECTIVE class, achieving a very low recall of just 0.09 before tuning.

In conclusion, this experiment clearly demonstrates that **effective feature representation** and **access to a large dataset** were the most decisive factors for success in this text classification task. The simplest model with the best features and the most data ultimately proved to be the most effective.